

Experiment 3

Module Name: Python BDD Restful Automations

Experiment Name: Selenium Page Object Model in the Python Behave Framework

Problem statement: Embedding raw Selenium WebDriver locators (like By.ID) directly inside Behave step definitions leads to brittle code. If the UI changes, developers have to hunt through dozens of step definition files to fix broken locators.

Objective: Debug and optimize a web application automations using behave framework and Gherkin language

Tools, software, and concepts used: Python 3, Selenium WebDriver, Behave BDD, Object-Oriented Programming (OOP), Page Object Model (POM).

Source code or implementation steps:

1. Create any Folder
2. Inside that folder create 'pages' Folder and 'features' folder
3. Inside that 'pages' folder create 'login_page.py' file
4. Inside that login_page.py file write this:

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class LoginPage:
    def __init__(self, driver):
        self.driver = driver
        self.url = "https://www.saucedemo.com/"
        self.username_locator = (By.ID, "user-name")
        self.password_locator = (By.ID, "password")
        self.login_button_locator = (By.ID, "login-button")

    def load(self):
        self.driver.get(self.url)

    def login(self, username, password):
        WebDriverWait(self.driver, 10).until(
            EC.presence_of_element_located(self.username_locator)
        ).send_keys(username)

        self.driver.find_element(*self.password_locator).send_keys(password)
        self.driver.find_element(*self.login_button_locator).click()
```

5. In the 'features' folder create a file 'pom_login.feature' and write this

Feature: Page Object Model Integration

Scenario: User logs in using POM architecture

Given the login page is loaded

When the user authenticates with standard credentials

Then the inventory dashboard should be displayed

6. In the 'features' folder create a new folder 'steps'

7. Inside 'steps' folder make a file 'pom_steps.py' and write this

```
from behave import given, when, then
from selenium import webdriver
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from pages.login_page import LoginPage

@given('the login page is loaded')
def step_impl(context):
    context.driver = webdriver.Chrome()
    context.driver.maximize_window()
    # Initialize POM and load page
    context.login_page = LoginPage(context.driver)
    context.login_page.load()

@when('the user authenticates with standard credentials')
def step_impl(context):
    # Use the POM login method directly
    context.login_page.login("standard_user", "secret_sauce")

@then('the inventory dashboard should be displayed')
def step_impl(context):
    WebDriverWait(context.driver, 10).until(
        EC.url_contains("inventory.html")
    )
    assert "inventory.html" in context.driver.current_url
    context.driver.save_screenshot("pom_success.png")
    context.driver.quit()
```

8. In the folder go to cmd and write this command

```
>>> python -m behave features/pom_login.feature
```

Output with clearly labelled screenshots:

```
F:\My Desktop\Project>python -m behave features/pom_login.feature
USING RUNNER: behave.runner:Runner
Feature: Page Object Model Integration # features/pom_login.feature:1

    Then the inventory dashboard should be displayed # features/steps/pom_steps.py:20 2.555s
    Then the inventory dashboard should be displayed # features/steps/pom_steps.py:20
1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
3 steps passed, 0 failed, 0 skipped
Took 0min 8.841s
```

Screenshot 1: Terminal execution of the POM integrated Behave scenario

Console Output:

```
1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
3 steps passed, 0 failed, 0 skipped
Took 0min 6.885s
```

Brief Result: The test executed successfully while maintaining a strict separation of concerns. Browser lifecycle was managed by Behave hooks (environment.py), UI interactions by POM (login_page.py), and assertions by step definitions.

Observation: The step definitions no longer contain any Selenium locators. They simply call readable methods like context.login_page.login().

Conclusion: Integrating POM with Behave is the gold standard for enterprise automation. It ensures the framework remains highly maintainable; if a developer changes an HTML ID, the QA engineer only updates one single variable in the Page Object class.